BigBird: Transformers for Longer Sequences

Cellula Internship — Week 3, Task 0: Research about the BigBird model
Heba Qasem

This research explains the BigBird Transformer model: why it was created, how its sparse attention works (window, global and random attention, implemented in blocks), how its computational complexity compares with BERT and Longformer, where it is useful, and what its main advantages, limitations and performance characteristics are.

Main source: Zaheer et al., *Big Bird: Transformers for Longer Sequences*, Google Research, NeurIPS 2020.

1. Motivation: the length problem of Transformers

Transformer models such as **BERT** changed natural language processing. Their key part is **self-attention**: to understand a word, the model looks at every other word in the text. In the sentence *"The firm for which Jacob worked sent him to New York"*, the word *him* looks at all the words and finds that *Jacob* is the most important one for its meaning.

The problem is the cost. If a text has **n** tokens, every token looks at **n** tokens, so attention computes **n × n** scores. Time and memory grow **quadratically**: when the text becomes 2 times longer, the work becomes 4 times larger. Because of this, BERT is limited to **512 tokens** (about one page).

Many important texts are much longer than 512 tokens: research papers, legal contracts, medical records, books, long web pages and DNA sequences. With BERT they must be cut into pieces, and the model can no longer connect information that is far apart (for example, a name on the first page and a pronoun on the third page).

BigBird's goal: keep the power of attention but make its cost grow **linearly** with the length, so that much longer texts (up to **4,096 tokens**, 8 times BERT's limit) fit on the same hardware.

2. Background: standard (full) self-attention

Each token is turned into three vectors:

- **Query (Q):** what this token is looking for.
- **Key (K):** what this token contains.
- **Value (V):** the information this token gives to the others.

The score between token *A* and token *B* is the dot product of *A*'s query and *B*'s key. Softmax turns the scores of each row into weights that add up to 1, and the new representation of *A* is the weighted sum of all values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

The term QK^T is the **attention matrix**: an **n × n** table with one score for every pair of tokens (row = the token that attends, column = the token it attends to). In full attention every cell is computed. The

same matrix can be drawn as a **graph**: every token is a node and every computed score is an edge. Full attention is a **complete graph** — every node is connected to every other node.

The next table shows how fast the memory for these score tables grows in **BERT-base** (12 layers × 12 attention heads, one 4-byte number per score, for a single text).

Sequence length n	Scores per table (n^2)	Memory for all 144 tables
512	262,144	0.14 GB
1,024	1,048,576	0.56 GB
2,048	4,194,304	2.25 GB
4,096	16,777,216	9.00 GB

The length grew **8 times** (512 → 4,096), but the memory grew **64 times** (0.14 GB → 9 GB) — and this is only the attention scores of one text, without the rest of the model. This is the **quadratic complexity $O(n^2)$** that BigBird addresses.

3. BigBird architecture and key innovations

BigBird keeps the Transformer architecture of BERT (in practice it starts from a pretrained **RoBERTa** checkpoint) and changes only **which scores are computed**. Its idea comes from graph theory: instead of a complete graph, use a **sparse graph** that has few edges but in which information can still travel between any two tokens in a few steps. Each token attends to three kinds of tokens:

3.1 Window (local) attention

Each token attends to its **neighbours** (the tokens just before and after it). Most of the meaning of a word comes from nearby words (*New* → *York*); this is called **locality**. In the attention matrix it is a band along the diagonal. Alone, it cannot connect distant words.

3.2 Global tokens

A few tokens are **global**: they attend to **all** tokens, and **all** tokens attend to them. They work like a **central station**: any two tokens can exchange information in two steps (token → global token → token), even if they are thousands of positions apart. There are two ways to choose them:

- **ITC (Internal Transformer Construction):** some **existing** tokens of the text become global (in the Hugging Face implementation: the first and the last block).
- **ETC (Extended Transformer Construction): extra** tokens are added only to be global (for example, the tokens of a question in question answering).

3.3 Random attention

Each token also attends to a few **randomly chosen** tokens. These random edges are **shortcuts** between distant parts of the text. In random graphs, the path between any two nodes stays short even when there are very many nodes (the "small world" idea, like "six degrees of separation"), so information spreads quickly across layers.

3.4 The combined pattern

BigBird attention = window + global + random. Every token attends to a **fixed** number of tokens that does not depend on the text length, so the cost becomes **linear: O(n)**. The next figure shows the four patterns and their combination for a text of 16 tokens (blue = computed score).

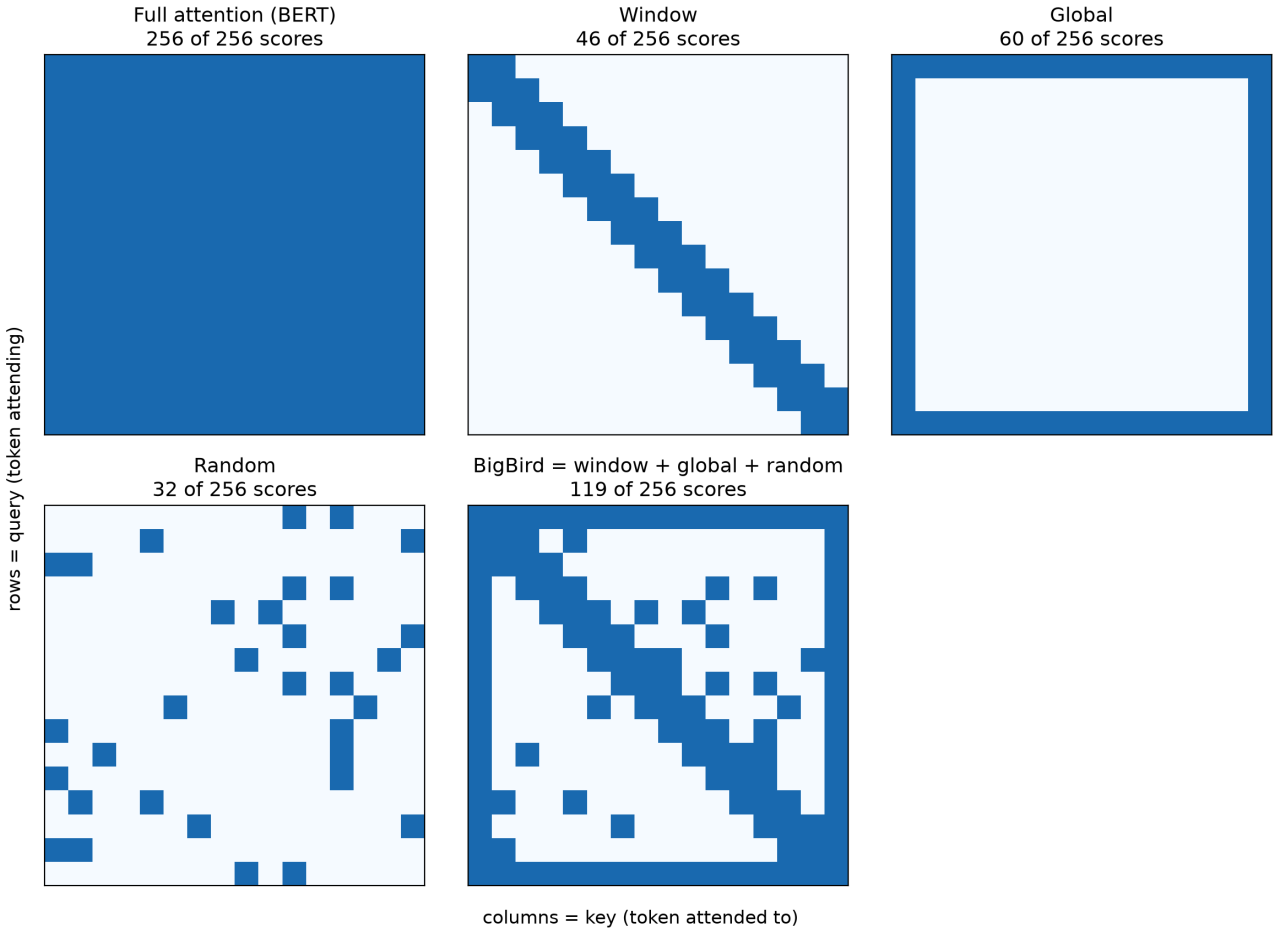


Figure 1. Attention patterns for 16 tokens (blue = computed score). BigBird combines window, global and random attention.

The same idea as a graph, on the sentence from the task. On the left, full attention connects every pair of words. On the right, BigBird keeps far fewer connections, but every word can still reach every other word through its neighbours, the global words (*The* and *York*) and the random shortcuts.

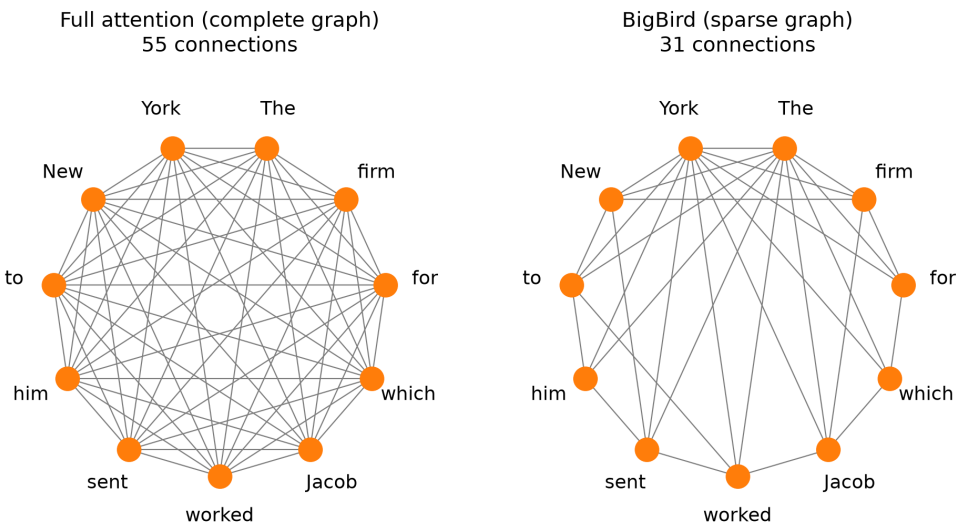


Figure 2. Attention as a graph. Full attention is a complete graph; BigBird is a sparse graph that stays connected.

3.5 Theoretical guarantees

The authors proved that BigBird keeps the theoretical power of full attention: it is a **universal approximator of sequence functions** and **Turing complete**. Their analysis shows that the **global tokens** are essential for these guarantees.

Experiments showed that window and random attention alone were not enough to reach BERT's results, which is why the global tokens are needed.

3.6 Block-sparse attention: making it fast on real hardware

GPUs and TPUs are very fast at multiplying large **dense** matrices, but slow at reading many scattered single values from memory. A sparse pattern computed token by token can therefore be slower than full attention, even with fewer operations.

BigBird solves this by **"blockifying"** the pattern: the sequence is split into **blocks of 64 consecutive tokens**, and attention is decided **between blocks**. Every piece of work becomes a small dense 64 × 64 matrix multiplication, which the hardware handles efficiently. With the default settings, each query block attends to:

Part	Key blocks	Tokens
Window: the block itself and its two neighbours	3	192
Global: the first and the last block	2	128
Random: randomly chosen blocks	3	192
Total	**8**	**512**

So every token attends to about **512 tokens, whatever the length of the text**; the first and last (global) blocks attend to everything. The next figure shows this pattern at block level for a text of 1,024 tokens (16 blocks); each cell is a full 64 × 64 block of scores.

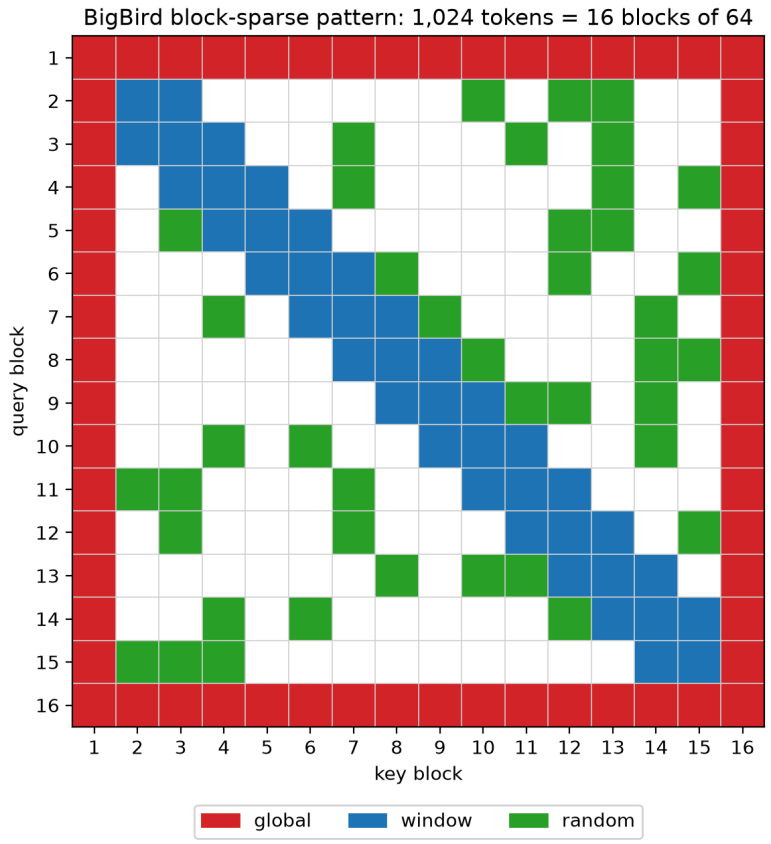


Figure 3. Block-sparse pattern for 1,024 tokens (16 blocks of 64). A middle query block attends to 8 of the 16 key blocks.

Practical consequences of the block design:

- The sequence length must be a **multiple of the block size** (64); shorter inputs are padded.
- For a text of **512 tokens** there are only 8 blocks, so attending to 8 blocks **is** full attention — there is no saving. Hugging Face therefore recommends normal full attention (`attention_type="original_full"`) for texts shorter than about **1,024 tokens**, and switches to it automatically when a text is too short for the sparse pattern.
- The Hugging Face implementation supports **ITC only**.

4. Computational complexity: BERT vs Longformer vs BigBird

Longformer (Beltagy, Peters and Cohan, Allen Institute for AI, 2020) solves the same problem with a similar idea:

- **Sliding window attention:** each token attends to its neighbours; the released model uses a window of **512 tokens** (256 on each side). The paper also proposes a *dilated* window with gaps, to cover a longer distance.
- **Global attention on task-specific tokens** chosen by the user: the `<s>` token for classification, the question tokens for question answering.
- **No random attention.**

Model	Each token attends to	Complexity
BERT	all *n* tokens	**$O(n^2)$** — quadratic
Longformer	window *w* + global *g*	**$O(n \cdot w)$** — linear
BigBird	window *w* + random *r* + global *g*	**$O(n \cdot (w + r + g))$** — linear

Since *w*, *r* and *g* are fixed numbers, Longformer and BigBird are both **linear, O(n)**. The next figure and table count the attention scores computed in one attention table with the default settings (Longformer window = 512; BigBird = 8 blocks of 64 per query block, global blocks attend to everything). BERT cannot read more than 512 tokens; its values above 512 show what full attention would cost.

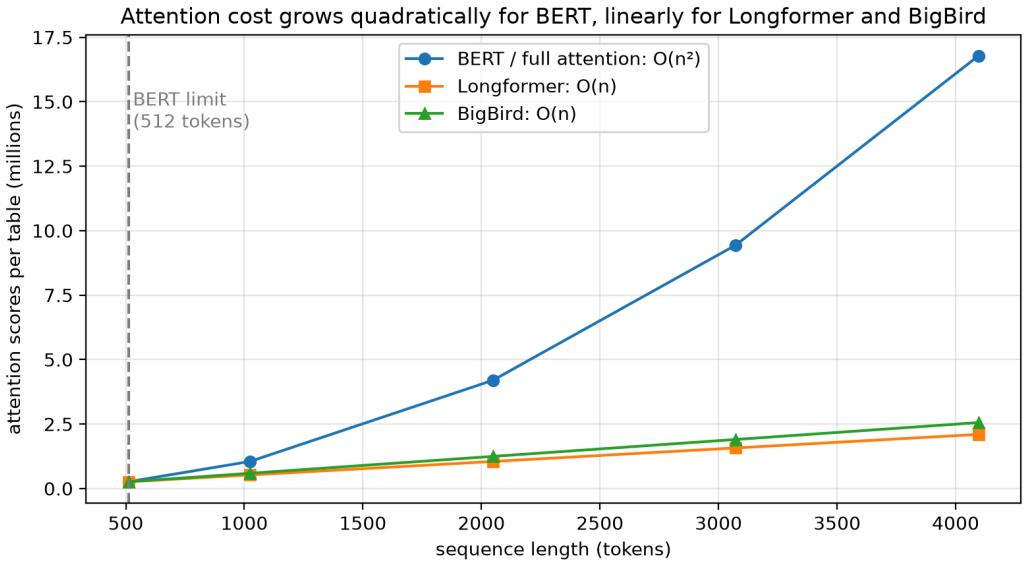


Figure 4. Attention scores per table vs sequence length. At 4,096 tokens, Longformer and BigBird compute about 12.5% and 15% of full attention.

Length	BERT (full)	Longformer	BigBird
512	262,144	262,144 (100%)	262,144 (100%)
1,024	1,048,576	524,288 (50%)	589,824 (56%)
2,048	4,194,304	1,048,576 (25%)	1,245,184 (30%)
4,096	16,777,216	2,097,152 (12.5%)	2,555,904 (15%)

Reading the results: at 512 tokens the three models do the same work; when the length doubles, BERT's cost is multiplied by 4 but Longformer's and BigBird's only by 2; at 4,096 tokens both compute about **one eighth** of full attention. Their costs are close — the real differences are in the design:

	Longformer	BigBird
Authors	Allen Institute for AI, 2020	Google Research, 2020
Window attention	✓ (512 tokens)	✓ (3 blocks = 192 tokens)
Global tokens	✓ chosen per task by the user	✓ fixed first/last blocks (ITC) or extra tokens (ETC)
Random attention	×	✓ (3 blocks)
Implementation	sliding window, token level	blocks of 64 tokens (GPU/TPU friendly)
Theoretical guarantees	—	universal approximator, Turing complete
Maximum length (released models)	4,096	4,096
Starting point	RoBERTa checkpoint	RoBERTa checkpoint

5. Applications for long documents

BigBird is useful whenever a task needs a long text and connections between distant parts of it:

- **Question answering on long documents:** the answer may be far from the question's keywords, or need two facts from different paragraphs (Natural Questions, HotpotQA, TriviaQA, WikiHop).
- **Long document classification:** for example classifying full research papers by topic.
- **Summarization of long documents:** research papers (arXiv, PubMed) and patents (BigPatent), using **BigBird-Pegasus**, an encoder-decoder model whose encoder uses BigBird attention.
- **Genomics:** DNA is a very long sequence of letters (A, C, G, T) where distant regions influence each other; BigBird was used for tasks such as finding promoter regions, the parts of DNA that switch genes on.
- **Practical uses:** legal contracts, medical records, technical reports, books and any input longer than 512 tokens. For RAG systems, a long-input model can also read larger chunks, so fewer connections are lost when cutting documents.

Pretrained checkpoints are available on Hugging Face, for example `google/bigbird-roberta-base` and `google/bigbird-pegasus-large-arxiv`.

6. Performance characteristics

Efficiency

- Time and memory grow linearly: each token attends to about 512 tokens, whatever the length of the text.
- BigBird reads up to **4,096 tokens**, 8 times more than BERT, on similar hardware. At this length, its attention needs about **one eighth** of the computation of full attention.
- For short texts (below about 1,024 tokens), there is no speed gain.

Results reported in the paper

- **Question answering on long documents:** better than RoBERTa (512 tokens) and Longformer, and the best results at that time on several benchmarks.

- **Summarization of long documents:** BigBird-Pegasus wrote better summaries than Pegasus for scientific papers and patents.
- **Classification of long documents:** better results, for example on full research papers.
- **Genomics:** better than earlier models on DNA tasks such as finding promoter regions.
- **Short texts:** only small gains (for example on short movie reviews), which shows that BigBird helps mainly when the input is long.

7. Advantages

1. **Linear complexity:** texts up to 8 times longer than BERT on the same hardware.
2. **Same theoretical power as full attention:** universal approximator and Turing complete, thanks to the global tokens.
3. **Better results on long-document tasks:** question answering, summarization, classification and genomics.
4. **Hardware friendly:** the block-sparse design turns sparse attention into small dense matrix multiplications.
5. **Easy to reuse:** pretrained models on Hugging Face, started from RoBERTa instead of training from scratch.

8. Limitations

1. **No benefit for short texts:** below about 1,024 tokens the sparse pattern saves almost nothing and adds overhead, so full attention is recommended.
2. **Some tasks need more layers ("no free lunch"):** in one layer, each token sees only part of the text, so information has to travel step by step. A few tasks that full attention solves in **one** layer need **many more layers** with sparse attention. This is true for all sparse attention methods, not only BigBird.
3. **More complex implementation:** the length must be a multiple of the block size, the blocks near the start and end need special handling, and the Hugging Face version supports only ITC.
4. **More settings to choose:** block size, number of random blocks and window size all affect speed and accuracy.
5. **Still a length limit:** the released models stop at 4,096 tokens; longer inputs (a whole book) still need to be split or handled with other methods.
6. **Fixed global tokens (ITC):** less flexible than Longformer, where the user chooses the global tokens for each task.

9. Conclusion

Full self-attention compares every token with every other token, so its cost grows quadratically and BERT is limited to 512 tokens. BigBird replaces the complete attention graph with a sparse one — **window** attention for local meaning, **global** tokens as central hubs and **random** attention as shortcuts — and implements it with **blocks** of 64 tokens so that it runs efficiently on GPUs and TPUs. The cost becomes linear, the input can grow to 4,096 tokens, and the theoretical power of full attention is kept. Compared with Longformer, which has a similar linear cost, BigBird adds random attention, a block-based implementation and theoretical guarantees, and it reported better results on

long-document question answering. Its main limits are that it does not help with short texts, that some tasks need more layers with sparse attention, and that its implementation is more complex.

References

1. M. Zaheer et al., *Big Bird: Transformers for Longer Sequences*, NeurIPS 2020. arXiv:2007.14062.
2. I. Beltagy, M. E. Peters, A. Cohan, *Longformer: The Long-Document Transformer*, 2020. arXiv:2004.05150.
3. J. Devlin et al., *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, NAACL 2019.
4. A. Vaswani et al., *Attention Is All You Need*, NeurIPS 2017.
5. Hugging Face blog, *Understanding BigBird's Block Sparse Attention* — huggingface.co/blog/big-bird
6. Hugging Face Transformers documentation: *BigBird* and *Longformer* model pages.